

# How to use the negotiation agent

A start-to-finish guide for running the agent, reading what it produces, and understanding every part of it. No prior context needed. If you can open a terminal and run one command, you can use everything here.

For the architecture overview, see the [README](#). For the pricing math in depth, see [CALCULATOR.md](#). For plugging the agent into the Oblsk platform, see [INTEGRATION.md](#). This document is the practical manual for how to run the agent.

---

## 1. What the agent does

You give it a creator (their recent view numbers) and a campaign (what you're selling and what you're willing to pay). It does two jobs:

1. **Prices the deal.** It works out what this creator is worth to you and turns that into three numbers: where to open, where to aim, and the most you'd ever pay. These three numbers are the *pricing ladder*, and every decision the agent makes is measured against them.
2. **Runs the conversation.** It reads each message the creator sends, decides the next move (open with an offer, hold firm, raise the offer, package a bundle, accept, hand off to a person, and so on), and writes the reply in plain, human language.

The split matters: the math decides *what* the deal should be, and the writing decides *how* to say it. The language model that writes the messages never picks the price or the move. If there's no API key, the agent still runs and still prices every deal; it just uses built-in message templates instead of a model.

Two things it will not do: it won't negotiate contract or legal terms (those go to a person), and it never tells the creator it's software.

---

## 2. Install

You need Python 3.10 or newer.

```
pip install -r requirements.txt      # numpy, scipy, matplotlib, pyyaml
pip install anthropic                # optional: lets a model write the messages
```

Check it works by running the tests. They run fully offline and take a few seconds:

```
python -m pytest
```

You should see all tests pass. If you don't have an API key set, the tests still pass; they're built to run without one.

On Windows the launcher is usually `py` instead of `python`. The examples below use `py demo.py`; use `python demo.py` if that's what your system has.

---

### 3. Your first run

The fastest way to see the agent work is a single simulated negotiation. A rule-based fake creator plays the other side, so the whole thing runs by itself:

```
py demo.py
```

You'll see the creator's messages, the agent's replies, and a short "why" line under each reply explaining the move. At the end there's a result summary: whether the deal closed, how many rounds it took, the final price, and the return on that spend.

To understand the prices before watching a negotiation, add `--explain`:

```
py demo.py --explain
```

This prints the calculation in plain English first — how many views the creator is expected to get, what a view is worth to you, and how those turn into the three ladder prices — and then runs a negotiation.

---

### 4. The pricing ladder (the three numbers everything hinges on)

Every negotiation runs on three prices. Once you understand these, the agent's behavior is predictable.

- **Anchor** — where the agent opens. Low but believable. It leaves room to move up so the creator feels they negotiated.
- **Target** — where the agent aims to land. This is the fee that still hits the campaign's return goal for a typical result.
- **Walk-away** — the ceiling. Above this price, an ordinary bad month for the creator would lose you money, so the agent never crosses it and never reveals it. It's the line used to judge whether a creator's ask is reasonable.

Here's how those three numbers get built, using the plain-English output from `--explain` as the example:

1. **Reach.** The agent simulates the creator's next video thousands of times, drawn from their real view history. This gives a bad day, a typical day, and a good day instead of a single guess. It matters because creator views are lopsided: most videos land near their usual number, and once in a while one runs far past it. Those rare hits are valuable, and a plain average hides them.
2. **Dollars.** Each view is worth some amount to you, based on how often a viewer becomes a customer and what a customer is worth. Multiply views by that per-view value and you get what a video is worth in dollars.
3. **Prices.** From that value and your return goal, the agent derives the anchor, target, and walk-away.

You'll sometimes see "target = walk-away" with a note that the creator's views are very tail-heavy. That's not a bug. It means this creator's value depends so much on rare viral hits that the safe

reading is to treat the aim and the ceiling as the same number.

---

## 5. The ways to run it

Everything below is `demo.py` with different flags. Run `py demo.py --help` to see them all at once.

### Talk to it yourself (- - chat)

You play the creator and type messages; the agent reads each one, prices the move, and replies live. This is the single best way to get a feel for it.

```
py demo.py --chat
py demo.py --chat --campaign examples/unest_campaign.yaml --median 150000
```

After each of your messages you'll see how the agent read it (interested? a question? an offer of a specific price?), the reply it drafted, the move it chose, and whether that reply would auto-send or wait for a human's OK. Type `quit` to stop.

This is the exact same read → price → write pipeline the Oblsk platform runs on real Gmail threads. There, email delivers the messages; here, you type them.

### Watch a full simulated negotiation (- - random)

A fresh, randomly-generated creator every run, negotiated end to end with the reasoning shown:

```
py demo.py --random          # new creator and full transcript each run
py demo.py --random --seed 42 # reproduce a specific one
```

Each run prints the seed. Pass that seed back with `--seed` to replay the exact same scenario.

### Run many at once and see the averages (- - count)

To see how the agent performs across a range of creators, run a batch. With `--quiet` you get one summary line per negotiation plus aggregate metrics at the end:

```
py demo.py --count 20 --quiet
```

The aggregate report shows the win rate, how often deals closed without needing a human, how often threads were escalated to a person, average rounds to close, average discount off the creator's first ask, and the average return on the closed deals.

### Shape a specific creator to test against

You can hand-build the fake creator instead of randomizing:

```
py demo.py --floor 2800 --opens ask --ask 3500
```

The knobs:

| Flag                 | What it sets                    | Default |
|----------------------|---------------------------------|---------|
| <code>--floor</code> | The lowest per-video price this | 2450    |

| Flag          | What it sets                                                         | Default  |
|---------------|----------------------------------------------------------------------|----------|
|               | creator will secretly accept                                         |          |
| - -opens      | How they start: question, interest, or ask                           | interest |
| - -ask        | Their opening per-video price, when - -opens ask                     | 3500     |
| - -counter    | How hard they haggle, from 0 (caves) to 1 (holds firm)               | 0.6      |
| - -bulk       | How much of a per-video discount they'll give for a multi-video deal | 0.06     |
| - -max-videos | The most videos they'll agree to do at once                          | 4        |

The floor is the *fake creator's* private minimum, used only by the simulation. The agent never sees it. The agent's own limit is the walk-away from the ladder.

## Turn off the approval gate (- -autonomy)

By default every outbound message waits for a human to approve it. To watch the agent send on its own (within the campaign's dollar ceiling), add - -autonomy:

```
py demo.py --autonomy
```

## Simulate a human rejecting a message (- -reject)

To see what happens when a reviewer rejects a proposed reply, name the move to reject. The agent hands the thread to a person, exactly as it would in production:

```
py demo.py --reject opening_flat
```

## 6. The moves the agent can make

Every reply the agent produces is one of these moves. The "why" line under each reply in the transcript names the move.

### Sent to the creator:

- **answer\_question** — answers a question about deliverables, timeline, usage rights, payment terms, and so on, straight from the campaign brief. It never quotes a price here; a money question routes to an offer instead.
- **opening\_flat** — opens with one flat price for a single video (the anchor), or raises to the target after the first rejection.
- **escalate\_bundle** — after a flat offer is turned down, packages several videos (or several formats) into one bundle at a fair per-video rate. More videos means more value to you, and

a bigger headline number to the creator.

- **add\_sweetener** — holds the price and adds a non-cash perk (faster payment, for example) to close a small remaining gap.
- **hold\_firm** — the creator followed up without a new offer, so the agent restates the price already on the table. It concedes nothing; a nudge is not a reason to lower the price.
- **propose\_call** — the creator wants a call, so the agent proposes the campaign's call windows and flags a teammate to send the invite and run it.
- **accept** — the creator's ask is at or under the ceiling, so the agent takes the deal.
- **soft\_close** — the creator isn't interested, so the agent closes warmly and leaves the door open. A second flat "no" with no counter-offer lands here too, instead of pestering someone who's already declined.

#### Handed to a person (not sent to the creator):

- **ask\_human** — the creator asked something the brief doesn't cover, or referenced a call the agent wasn't on. The agent pauses and asks the team for exactly what it needs rather than guessing. It keeps the thread; it just needs an answer.
  - **escalate\_human** — an unreasonable ask, an exhausted round budget, or a message negotiating contract terms hands the whole thread to a person.
  - **pause\_human** — a teammate sent a message in the thread, so the agent stands down until it's caught up.
- 

## 7. Campaign configuration

Everything specific to a campaign lives in one YAML file: what the agent says, what it's willing to pay, and how it behaves. Change campaigns by changing the file — no code edits.

Use one with `--campaign`:

```
py demo.py --campaign examples/unest_campaign.yaml --count 10 --quiet
```

`examples/unest_campaign.yaml` is a complete, commented example built from a real campaign. Open it and you'll find four sections:

- **brief** — the facts the agent answers questions from: the brand, the product, deliverables, timeline, usage rights, exclusivity, payment terms. Anything the agent tells a creator comes from here.
- **economics** — the two numbers that price every deal: how often a view turns into a customer (`conversion_rate`) and what a customer is worth (`ltv_usd`). These drive the whole ladder, so calibrate them against real closed deals before trusting the dollar figures.
- **negotiation** — the full stance: the return goal, how aggressively to open, bundle size, the dollar amount above which a human must sign off, and when the team can take calls.
- **us\_aliases** — the names, emails, or domains that identify your side of an email thread, so

the replay tool (below) knows which messages are yours.

If you mistype a key, the file fails to load with a clear error naming the bad key, so a typo can't silently fall back to a default stance.

---

## 8. Testing the agent against real threads

Before letting the agent near a live conversation, you can check it against negotiations your team already handled. This is `--replay`.

Paste a thread — a raw Gmail thread works as-is, with headers, signatures, quoted replies, and footers — and the agent walks through it message by message. At each creator message it shows how it read the message, the move it would have made, the reply it would have drafted, and why, lined up next to what your team actually sent. Nothing is ever sent anywhere.

```
py demo.py --campaign examples/unest_campaign.yaml \  
  --replay examples/unest_thread.txt --median 150000
```

`--median` is the creator's typical per-video view count. It sets the size of the deal the agent is pricing, so pass the creator's real number to make the comparison fair.

`examples/unest_thread.txt` is a real agency-run negotiation. On it the agent opens within \$100 of the team's real opener, accepts the same price the team agreed to, holds firm on the "just following up" nudges, proposes the intro call when the manager offers times, and escalates the contract-revision email to a human.

### Sparring: have a model play the creator's manager

To stress-test the back-and-forth against something that phrases things like a real person, have Claude role-play the creator's manager and negotiate against the agent. This one needs an API key.

```
py demo.py --campaign examples/unest_campaign.yaml \  
  --spar "floor is $2,600/video; open by asking $4,500; cite her reach;  
concede slowly; suggest a call once"
```

The text you pass is the manager's private playbook. The transcript shows every exchange with the agent's move and reasoning, so a person can grade the conversation.

---

## 9. Turning on the language model

Without an API key, the agent runs entirely on templates and keyword rules. That covers the tests, the demo, and any offline use. It still prices every deal correctly; the messages are just more formulaic.

Set an API key and the model takes over three jobs:

- **Writing outbound messages** — it drafts each reply from the agent's decision and the thread so far, making it sound like a real account manager. If a draft ever drops the price the calculator set, the agent throws it out and uses the template, so the numbers are always exactly right.

- **Answering open-ended questions** — it answers from the campaign brief in natural language, and is blocked from ever quoting a rate.
- **Reading messy real-world messages** — it turns a creator's free-text reply into a clean read of their intent and any price they named.

To enable it:

```
export ANTHROPIC_API_KEY=...          # your key
export OBLSK_LLM_MODEL=claude-opus-4-8 # optional: override the default model
```

To force offline mode even with a key present (useful for tests and demos):

```
export OBLSK_NO_LLM=1
```

The model makes the agent sound human and read messy text. It never sets a price or chooses a move; the calculator and the decision logic always do that.

---

## 10. Running it as a service (for the Oblsk platform)

The platform already handles Gmail, message classification, and the approval UI. What it needs from a negotiator is one thing: given a thread, what's the next reply? The agent serves exactly that over HTTP.

```
py -m oblisk_negotiator.service --campaign examples/unest_campaign.yaml --port 8788
```

- `POST /suggest` takes the thread and the creator's view numbers, and returns the drafted message, the move, the ladder, and whether a human needs to approve or act.
- `GET /health` returns a quick status check.
- Set `OBLSK_SERVICE_TOKEN` to require a bearer token on every request.

The service holds no state of its own. It reads the whole situation from the thread each time, so a person can jump into a thread at any point and the agent picks right back up. The wiring details and the exact request and response shapes are in [INTEGRATION.md](#).

---

## 11. Pausing and resuming

A negotiation can run over several days. The agent's state — where it is in the conversation, what's been offered, how many rounds have passed — saves to a plain text format and loads back exactly, so a thread can pause overnight and resume with nothing lost. The same history is also kept as an append-only event log, so you can reconstruct everything that happened in order.

When used as the platform service (section 10), there's nothing to save or load at all: the thread itself is the record, and the agent rebuilds its state from the thread on every request.

---

## 12. Common questions

**Do I need an API key to try it?** No. Everything runs offline with templates. The key only changes how human the messages sound and how well it reads unusual phrasing.

**Will it ever pay more than it should?** No. Every offer is measured against the walk-away ceiling, and the agent never crosses it, even after rounding to a clean number.

**What if the creator asks something the brief doesn't answer?** It pauses and asks your team the specific question rather than making something up.

**What if a teammate replies in the middle of a thread?** The agent notices and stands down until it's caught up, so it never talks over a person.

**Can I reproduce a run I saw?** Yes. Every run prints its seed; pass it back with `--seed`.

**Where do the prices come from?** The campaign's economics (conversion rate and customer value) and the creator's view history. Change either and the ladder changes. Run `py demo.py --explain` to see the full calculation in words.

---

## 13. Quick reference

```
# setup
pip install -r requirements.txt
pip install anthropic                # optional model support
python -m pytest                     # verify, runs offline

# see it work
py demo.py                          # one simulated negotiation
py demo.py --explain                 # show the price calculation first
py demo.py --chat                    # you play the creator
py demo.py --random                  # a new random creator each run
py demo.py --count 20 --quiet        # 20 runs + aggregate metrics

# with a real campaign
py demo.py --campaign examples/unest_campaign.yaml --random
py demo.py --campaign examples/unest_campaign.yaml \
    --replay examples/unest_thread.txt --median 150000

# as the platform service
py -m oblsk_negotiator.service --campaign examples/unest_campaign.yaml --port
8788

# environment variables
ANTHROPIC_API_KEY=...               # turn on the model
OBLISK_LLM_MODEL=...                 # override the model (default claude-opus-4-8)
OBLISK_NO_LLM=1                     # force offline mode
OBLISK_SERVICE_TOKEN=...             # require a bearer token on the service
```